3. Particle Swarm Optimization

3. PARTICLE SWARM OPTIMIZATION (PSO)

Inspiration in the social interaction between species

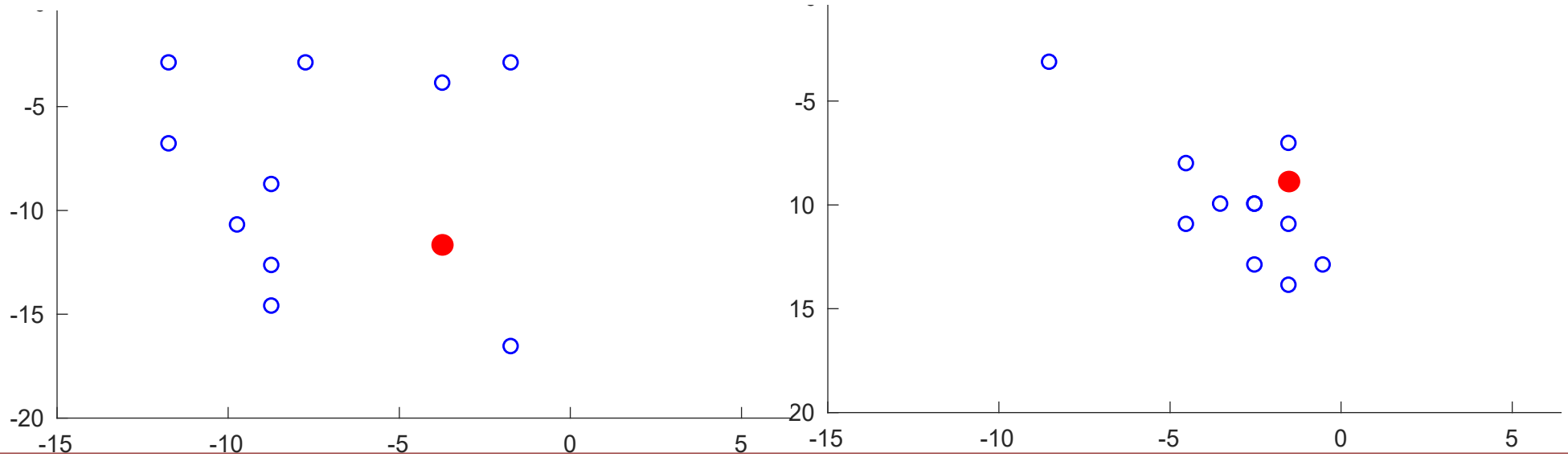
PSO is an optimisation technique based on the movement and intelligence of certain animal populations, such as schools of fish, flocks of birds, etc.

- PSO uses a certain number of agents (particles) that simulate the movement of a population in search of the best solution.
- Each particle is treated as a point in N-dimensional space that adjusts its motion according to the experience of its own movement, as well as the experience of other particles.

3.1 Basis

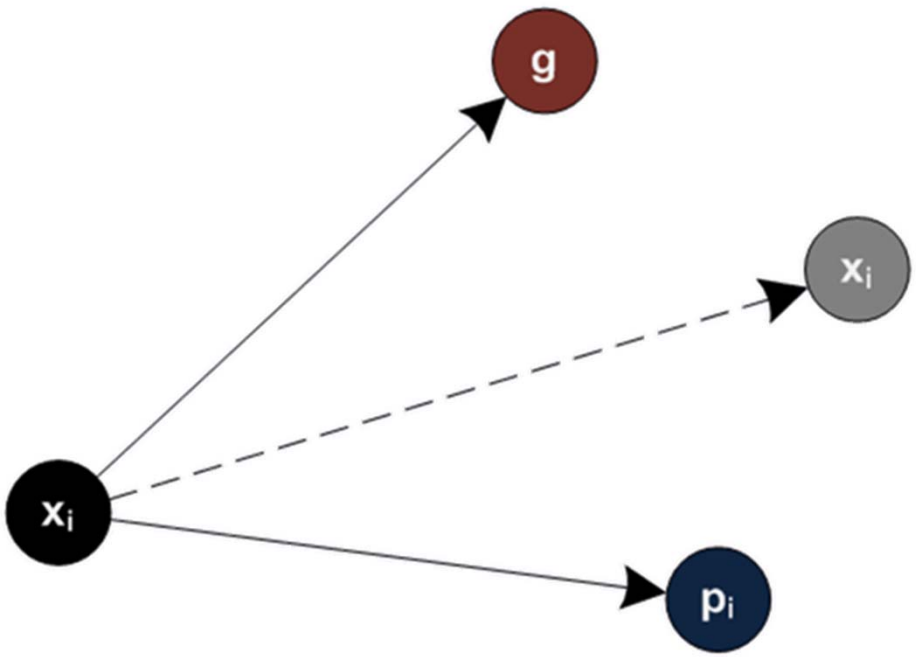
- **Personal best, *pbest*:** Each particle stores its coordinates in the state space. They are associated with the best solution (fitness) achieved so far by that particle.
- **Global optimal, *gbest*:** The best value obtained by a particle belonging to the whole population.

The basic process consists of guiding each particle towards the ***pbest*** and ***gbest*** locations, by changing the speed of the movement.



3.1 Basis

- A particle moves towards the best position in the search space, remembering:
  - its personal best location reached so far
  - the global best location of a particle in the whole population



3.2 PSO Algorithm

1. Generate randomly a population of different particles with different velocities.
2. In each iteration *k:*
   1. Evaluate fitness of each particle, *i*
   2. Determine each p_{best} and the global g_{best}:

      if fitness(p) better than pbest fitness

      then $p_{best}=p$

      g_{best} *= the best of all* p_{best}
   3. Update velocity and location of each particle, *i*, according to:
      - Current velocity V_i^k
      - Current position S_i^k
      - Distance between the current position and pbest $(S_{pbest_i} - S_i^k)$
      - Distance between the current position and gbest $(S_{gbest} - S_i^k)$

3.2 PSO Algorithm

$$V_i^{k+1} = wV_i^k + c_p\, r_p(S_{pbest_i} - S_i^k) + c_g\, r_g(S_{gbest} - S_i^k)$$

$$S_i^{k+1} = S_i^k + V_i^{k+1}$$

- Parameters:
  - S_i^k Position of the particle i in the iteration k
  - V_i^k Velocity of the particle i in the iteration k
  - S_{gbest} Best global position
  - $S_{pbest\ i}$ Best personal position of the particle i
- Coefficients:
  - Inertial constant ***w***, to establish the influence of the previous solution.
  - Accelerating constants **(c_p, c_g)** to establish the influence of the best particular and global.
  - Random numbers **(r_p, r_g)**: to avoid local optima.

3.3 Updates

In each iteration, the particle's behaviour is a trade-off among three possibilities:

$$V_i^{k+1} = wV_i^k + c_p\, r_p(S_{pbest_i} - S_i^k) + c_g\, r_g(S_{gbest} - S_i^k)$$

$$S_i^{k+1} = S_i^k + V_i^{k+1}$$

3.4 PSO formulation for binary representations

- In **Binary PSO**, the solution must be a vector of **bits (0/1)**, so particles cannot move by simply adding velocities.

$$X_i = (0, 1, 0, 0, 0, 1)$$

- The key idea is:
  - In Binary PSO, the velocity no longer represents a physical displacement, but the **probability that a bit takes value 1**.

$$V_i = (2.8, 1.7, -0.9, -0.1, 0.3, 0.1)$$

  - Instead of directly updating positions using addition, the velocity is transformed into a probability through a sigmoid function:

$$S(v) = \frac{1}{1 + e^{-v}}$$

3.4 PSO formulation for binary representations

- Suppose the search space is $S = \{0, 1\}^D$, and the objective function *f* is to be maximized, i.e., max *f(x)*, then the ith particle of the swarm can be represented by a D – dimensional vector,

$$X_i = (x_{i1}, x_{i2}, \ldots, x_{iD}),\ \ x_{id} \in \{0, 1\},\ \ d = 1,2,\ldots,D.$$

- The velocity (position change) of this particle can be represented by another D-dimensional vector

$$V_i = (v_{i1}, v_{i2}, \ldots, v_{iD}),\ \ v_{id} \in [-Vmax, Vmax],\ d = 1, 2, \ldots, D$$

and Vmax is the maximum velocity.

- Previously visited best position of the ith particle is denoted as

$$P_i = (p_{i1}, p_{i2}, \ldots, p_{iD}),\ \ p_{id} \in \{0,1\},\ d = 1,2,\ldots,D.$$

3.4 PSO binary representation

- Define **g** as the index of best performer in the swarm and $\mathbf{p_{gd}}$ as the swarm best, then the swarm is manipulated according to the following two equations:
  - Velocity update

$$v_{id}^{k+1} = wv_{id}^{k} + c_p r_p (p_{id} - x_{id}) + c_g r_g (p_{gd} - x_{id})$$

  - Position update

$$x_{i,j} = \begin{cases} 1 & \text{if } rand() < S(v_{i,j}) \\ 0 & \text{otherwise} \end{cases}$$

Therefore:

- *large positive velocity → high probability of bit = 1*
- *large negative velocity → high probability of bit = 0*
- *velocity near zero → random behavior*

3.4 PSO binary representation

where

- d = 1,2...D; i = 1,2...N, and N is the size of the swarm;
- c_p and c_g are constants, r_p, r_g are random numbers, uniformly distributed in [0,1].
- S (v_{id}) is a sigmoid function, defined as

$$S(v) = \frac{1}{1 + e^{-v}}$$

3.5 PSO para permutaciones de enteros: Conceptos

- **Posición de una partícula**

La posición de un partícula corresponde a una posible solución al problema, por tanto está constituida por los N posibles valores enteros. Por ejemplo, para N=8

S_i=[3 4 5 2 1 6 7 8]

3.5 PSO para permutaciones de enteros: Conceptos

- **Velocidad de una partícula**

Conjunto de intercambios aplicable a una partícula y que generan una nueva posición

Se representa por una lista de pares de enteros (i,j) que indican el intercambio a realizar sobre los elementos de S_i

Por ejemplo, si aplicamos la velocidad V=(4, 1) a

S_i=[3 4 5 2 1 6 7 8]

obtendremos

S_i=[2 4 5 3 1 6 7 8]

3.5 PSO para permutaciones de enteros: Operaciones

- **Suma de una posición y una velocidad**

Aplicación sucesiva de pares de V a Si.

Por ejemplo, siendo V = [(8, 1) (2,2) (7,3)]

La aplicación a Si=[3 4 5 2 1 6 7 8]

Da lugar a:

[3 4 5 2 1 6 7 8] → [8 4 5 2 1 6 7 3] (8,1)

[8 4 5 2 1 6 7 3] → [8 4 5 2 1 6 7 3] (2,2)

[8 4 5 2 1 6 7 3] → [8 4 7 2 1 6 5 3] (7,3)

3.5 PSO para permutaciones de enteros: Operaciones

- **Diferencia entre posiciones** (Sk – Sj)

  Evaluación de los movimientos necesarios para pasar de la posición Sj a la Sk. Se realizan posición a posición.

  Al restar dos posiciones el resultado es una lista velocidad, es decir, una lista de pares de intercambios sobre Sj.

  Por ejemplo:

  Sk= [3 4 5 2 1 6 7 8]

  Sj= [2 4 5 7 1 6 8 3]

  [2 4 5 7 1 6 3 8] (7,8)

  [2 4 5 3 1 6 7 8] (4,7)

  [3 4 5 2 1 6 7 8] (1,4)

3.5 PSO para permutaciones de enteros: Operaciones

- **Multiplicación de un coeficiente por una velocidad**

Probabilidad de elegir alguno de los términos que definen la velocidad

Ejemplo:

0.5 * [(1,8) (2,6) (4,3)]

para la secuencia de valores aleatorios 0.2, 0.5, 0.8

[(4,3)]

sólo se aplica el intercambio (4,3) cuyo valor aleatorio: 0.8 supera el valor del coeficiente que es 0.5

3.5 PSO para permutaciones de enteros: Operaciones

- **Suma de velocidades (Vk+Vj)**

  Aplicación sucesiva de diferentes movimientos de intercambio

  La suma de dos velocidades consiste en ir solapando los pares de las listas de dichas velocidades, de manera que se obtiene una nueva lista de pares:

  - Para que se produzca un nuevo par, deben coincidir los valores final e inicial de los pares involucrados, es decir, los pares (i ,j) y (j, k) se solapan formando el par (i, *k*)
    - Si dos pares no se solapan, el resultado será el primer par
    - Si las listas son de distinto tamaño, completar con la de mayor longitud

  Vk=[(1,1) (2,5) (5,4) (1,3)] **Vj**=[(2,2) (5,1) (4,3) (2,4)]

  Vk+Vj= [(1,1) (2,1) (5,3) (1,3)]

3.6 Summarizing

- Each agent or particle:
  - has a position (candidate solution)
  - has a velocity (to change the position more or less quickly)
  - remembers its best personal position
  - remembers the best global position
  - knows how to calculate next position and evaluation function